

Object-Oriented Programming in Java

Emily Lucia Antosch

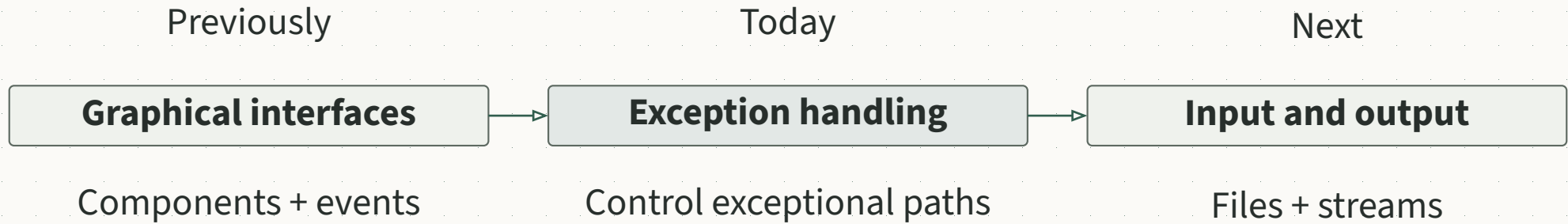
18.09.2026

Today's route

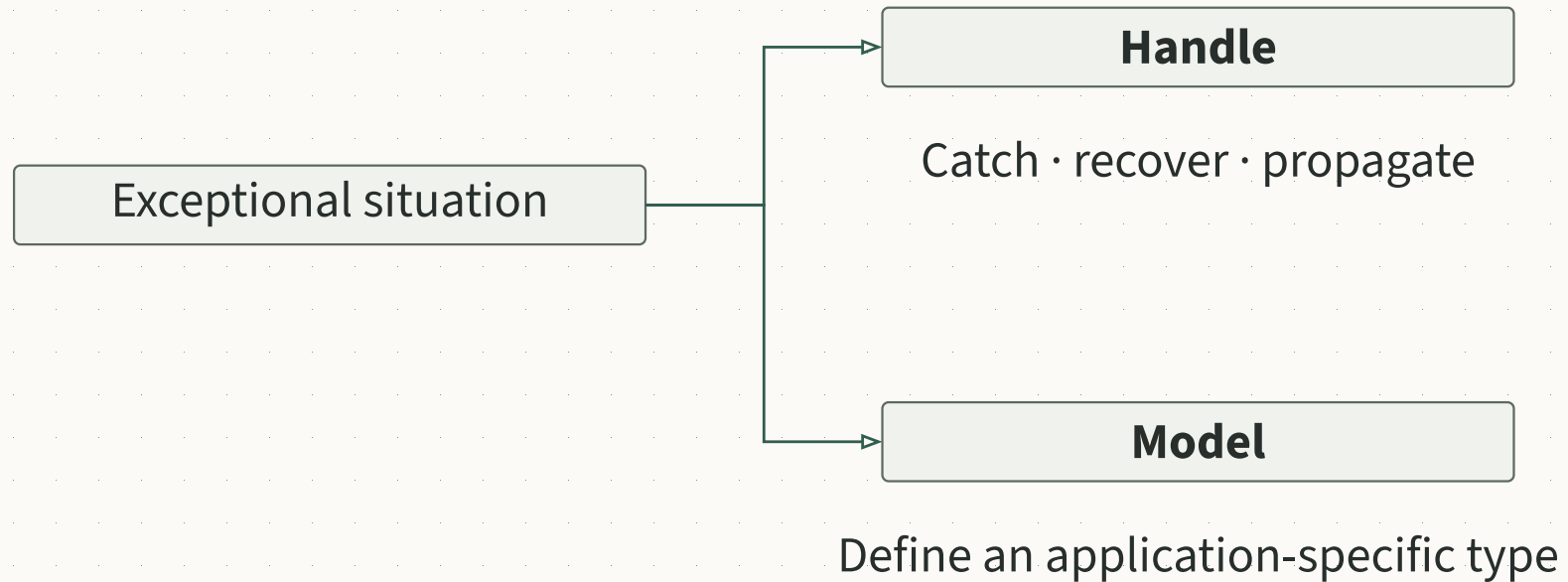
1. Introduction	2
2. Exception Handling	5
3. Throwing exceptions	11
4. Catching exceptions	16
5. Defining custom exceptions	36
6. License Notice	46

1. Introduction

From Interfaces to Exception Handling



Handle Failures, Define Their Meaning



2. Exception Handling

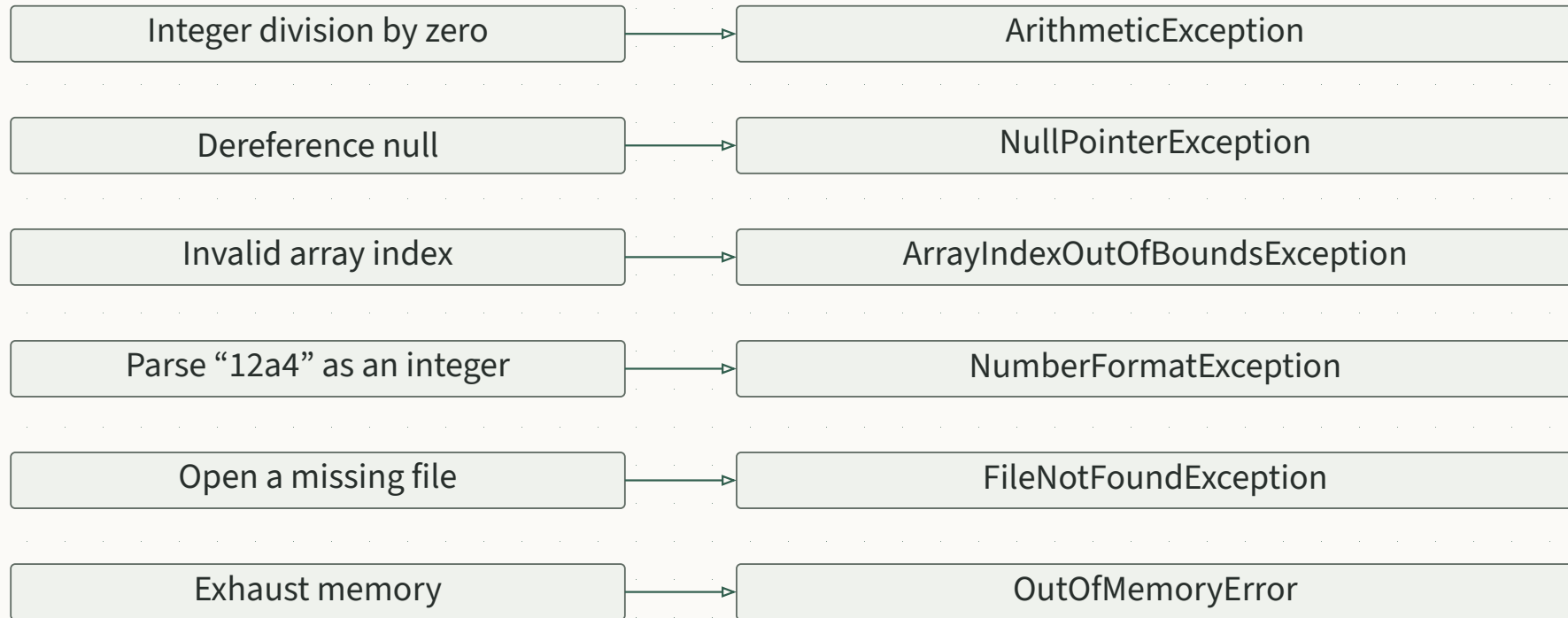
Predict the Output

What is printed for (3, 2)? What changes for (7, 0)?

```
1 public class ProvokeException {  
2     public static void main(String[] args) {  
3         printRatio(3, 2);  
4         System.out.println("Exiting main()");  
5     }  
6     public static void printRatio(int a, int b) {  
7         int ratio = a / b;  
8         System.out.println("Ratio = " + ratio);  
9     }  
10 }
```

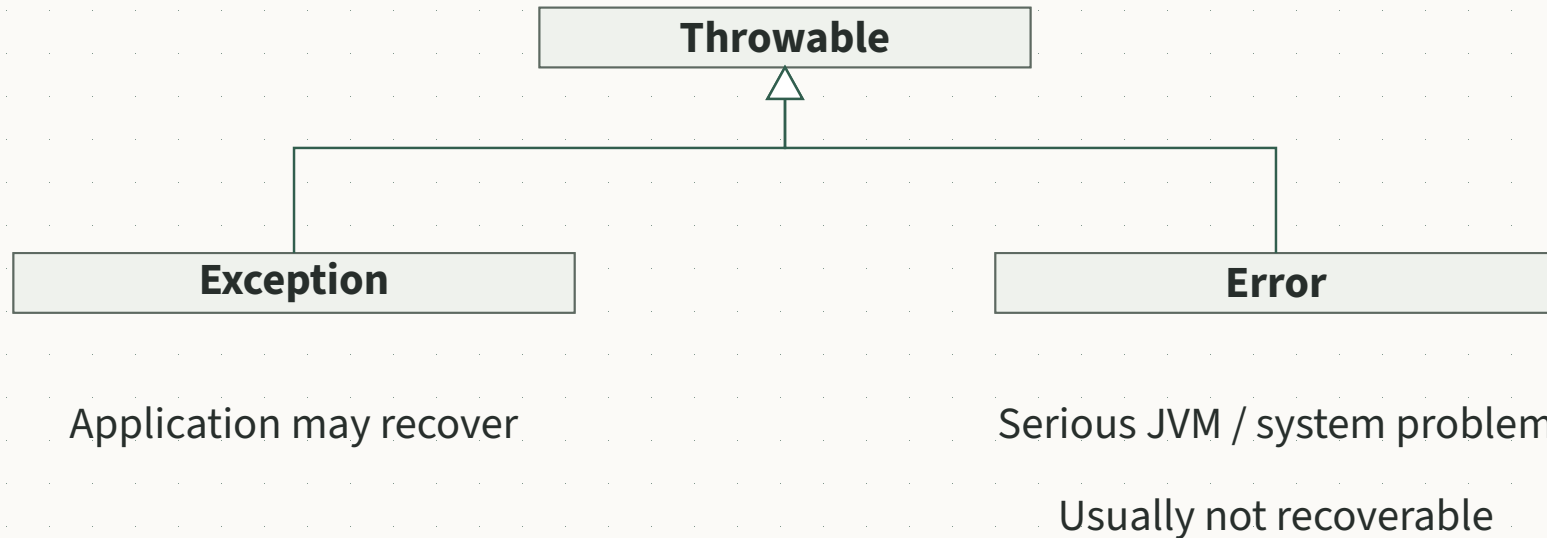
Java

What Can Fail?



Which failures could the application recover from?

Throwable: Exceptions and Errors



Both are throwable objects • Error can be caught, but rarely should be

Spot the Exception

Evaluate each snippet independently.

```
1 int[] array = {1, 2, 3, 4};  
2 System.out.println(array[4]);
```

Java

```
1 String message = null;  
2 System.out.println(message.length());
```

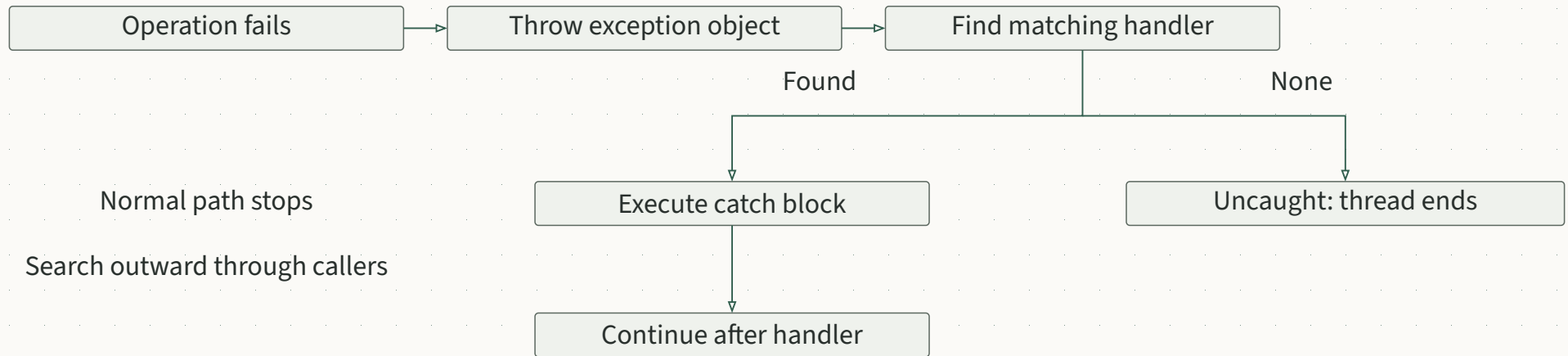
Java

```
1 int code = Integer.parseInt("12a4");
```

Java

An uninitialized local variable would be a compile-time error, not a null reference.

Throw, Search, Handle



3. Throwing exceptions

Throw an Exception Object

Java throws exceptions for failures such as integer division by zero. Your code can also throw explicitly:

```
1 throw new ArithmeticException("Division by zero");
```

Java

Or create the object first:

```
1 ArithmeticException problem =  
2     new ArithmeticException("Division by zero");  
3 throw problem;
```

Java

The optional message explains the failure.

Task: Reject a Zero Divisor



Task 1

Throw `ArithmeticException` before dividing if `b` is zero. Include a message.

```
1 public static void printRatio(int a, int b) {  
2     int ratio = a / b;  
3     System.out.println("Ratio = " + ratio);  
4 }
```

Java

An Explicit Guard

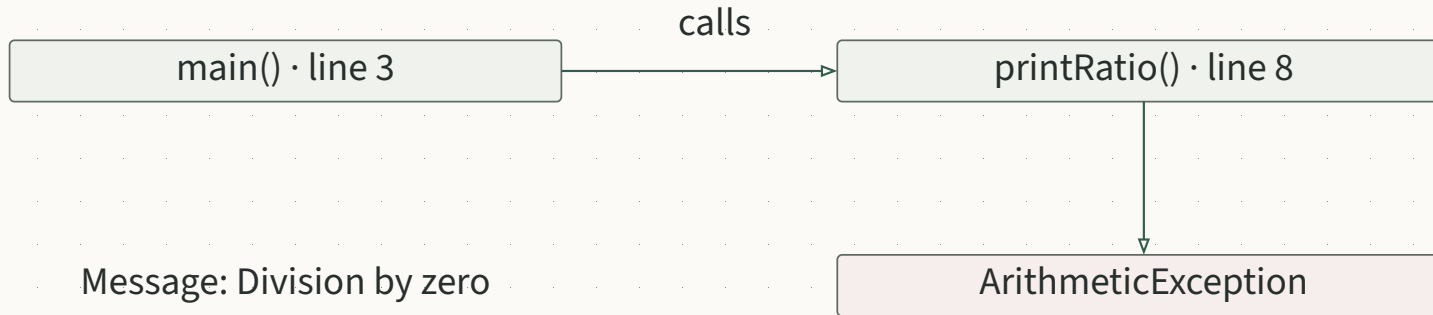
```
1  public class ThrowException {
2      public static void main(String[] args) {
3          printRatio(3, 0);
4          System.out.println("Exiting main()");
5      }
6      public static void printRatio(int a, int b) {
7          if (b == 0) {
8              throw new ArithmeticException("Division by zero");
9          }
10         System.out.println("Ratio = " + (a / b));
11     }
12 }
```

Java

The final print statement in main is never reached.

Read a Stack Trace

```
1 Exception in thread "main" java.lang.ArithmeticException: Division by zero
2   at ThrowException.printRatio(ThrowException.java:8)
3   at ThrowException.main(ThrowException.java:3)
```



Trace order: throw site first, then its callers

4. Catching exceptions

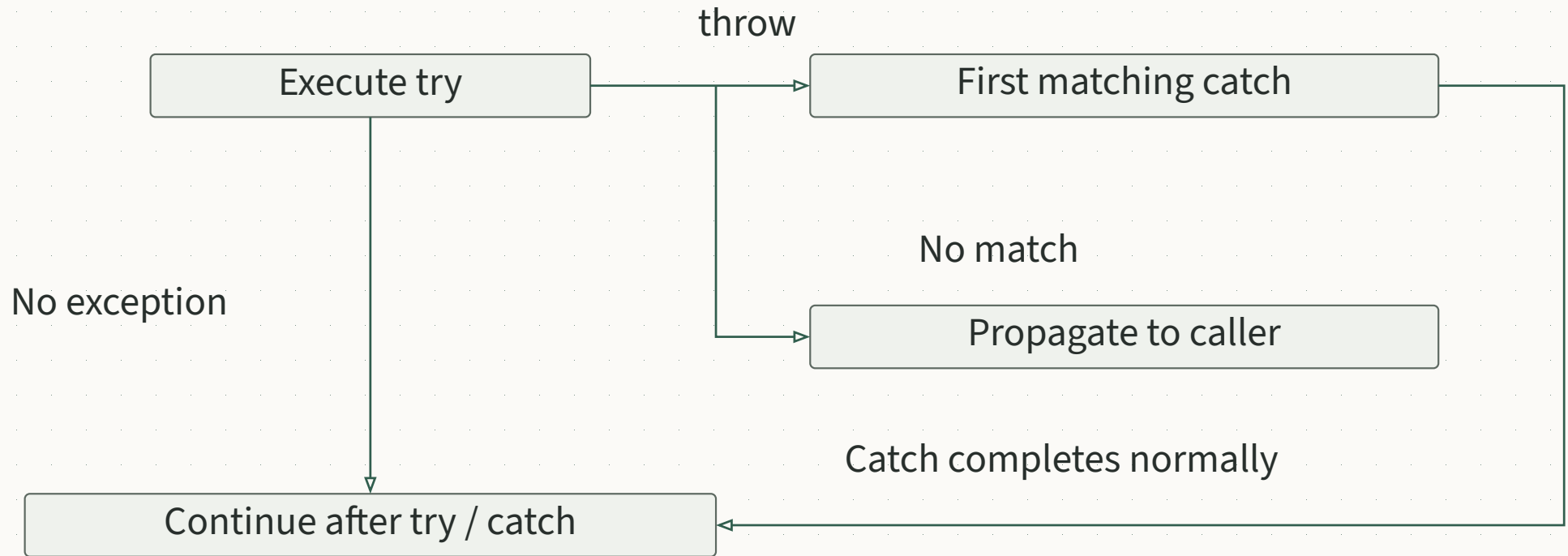
Catch a Matching Type

```
1 try {  
2     // Code that may throw.  
3 } catch (ArithmeticException e) {  
4     // Handle the failure.  
5 }  
6 // Continue if try or catch completes normally.
```

Java

A catch matches its declared type and subclasses. Throwing skips the rest of the try block.

Follow the Control Flow



Task: Keep the Caller Running



Task 2

Catch the division failure inside `printRatio`. Let `main` reach its final print statement.

```
1 public static void printRatio(int a, int b) {  
2     int ratio = a / b;  
3     System.out.println("Ratio = " + ratio);  
4 }
```

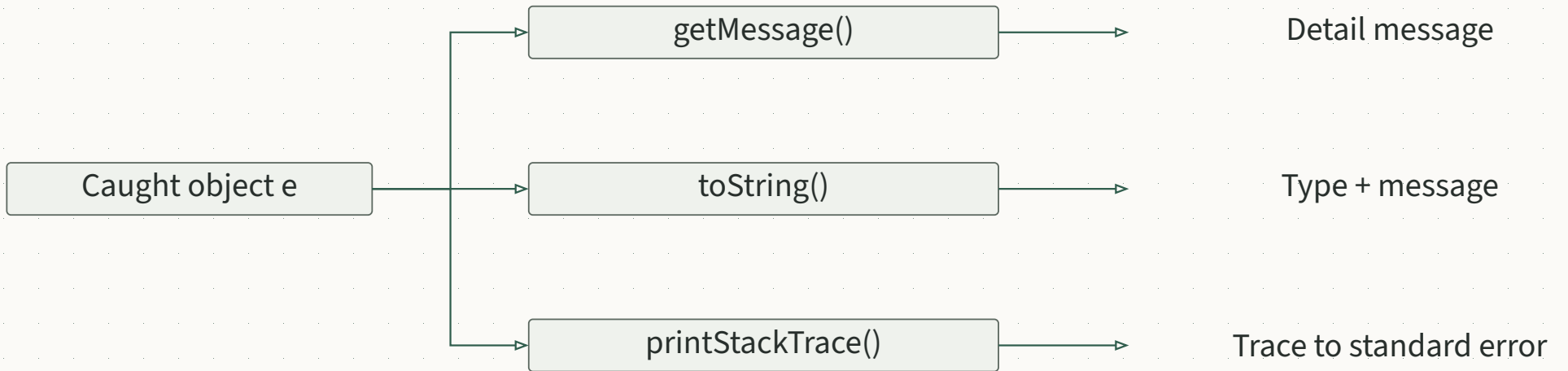
Java

Handle the Failure Locally

```
1  public static void printRatio(int a, int b) {  
2      try {  
3          int ratio = a / b;  
4          System.out.println("Ratio = " + ratio);  
5      } catch (ArithmeticException e) {  
6          System.out.println("Caught in printRatio()");  
7          System.out.println(e.getMessage());  
8          System.out.println(e.toString());  
9      }  
10     System.out.println("Exiting printRatio()");  
11 }
```

Java

Inspect the Exception Object



Catch Locally: Is Zero a Valid Result?

```
1  public static void main(String[] args) {
2      System.out.println("Ratio = " + getRatio(3, 0));
3      System.out.println("Exiting main()");
4  }
5  public static int getRatio(int a, int b) {
6      int ratio = 0;
7      try {
8          ratio = a / b;
9      } catch (ArithmeticException e) {
10         System.out.println("Caught in getRatio()");
11     }
12     System.out.println("Exiting getRatio()");
13     return ratio;
14 }
```

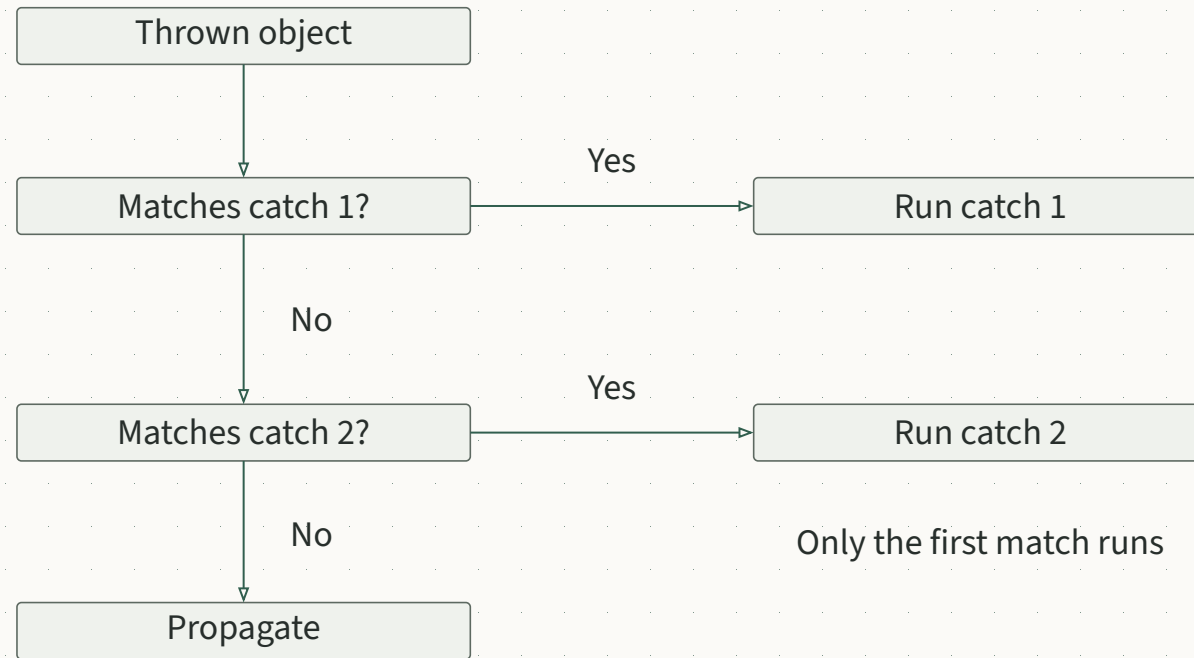
Java

Catch in the Caller

```
1  public static void main(String[] args) {  
2      try {  
3          int ratio = getRatio(3, 0);  
4          System.out.println("Ratio = " + ratio);  
5      } catch (ArithmeticException e) {  
6          System.out.println("Caught in main()");  
7      }  
8      System.out.println("Exiting main()");  
9  }  
10 public static int getRatio(int a, int b) {  
11     int ratio = a / b;  
12     System.out.println("Exiting getRatio()");  
13     return ratio;  
14 }
```

Java

Choose the First Matching Catch



Only the first match runs

Subclasses before superclasses · unreachable catch blocks do not compile

Put Specific Types First

```
1 try {  
2     // Code that may throw.  
3 } catch (NumberFormatException e) {  
4     // Invalid numeric text.  
5 } catch (IllegalArgumentException e) {  
6     // Other invalid arguments.  
7 }
```

Java

NumberFormatException extends IllegalArgumentException. Reversing these catches makes the first case unreachable.

Two Failures, One Evaluation Order

Which failure happens first? What is printed?

```
1  static int recursiveIncrease(int i) {  
2      return recursiveIncrease(i + 1);  
3  }  
4  public static void main(String[] args) {  
5      int[] a = new int[4];  
6      try {  
7          a[4] = recursiveIncrease(7);  
8      } catch (ArrayIndexOutOfBoundsException e) {  
9          System.out.println("Caught invalid index");  
10     }  
11     System.out.println("Exiting main()");  
12 }
```

Java

Catch Either Failure Type

Add a catch for `StackOverflowError`. Use `recursiveIncrease` from the previous slide.

```
1  public static void main(String[] args) {
2      int[] a = new int[4];
3      try {
4          a[4] = recursiveIncrease(7);
5      } catch (ArrayIndexOutOfBoundsException e) {
6          System.out.println("Caught invalid index");
7      } catch (StackOverflowError e) {
8          System.out.println("Caught StackOverflowError");
9      }
10     System.out.println("Exiting main()");
11 }
```

Java

For this assignment, the right-hand side runs before the bounds check. Only `StackOverflowError` occurs. Catching it is a teaching example, not a recovery strategy.

An Exception Inside a Catch

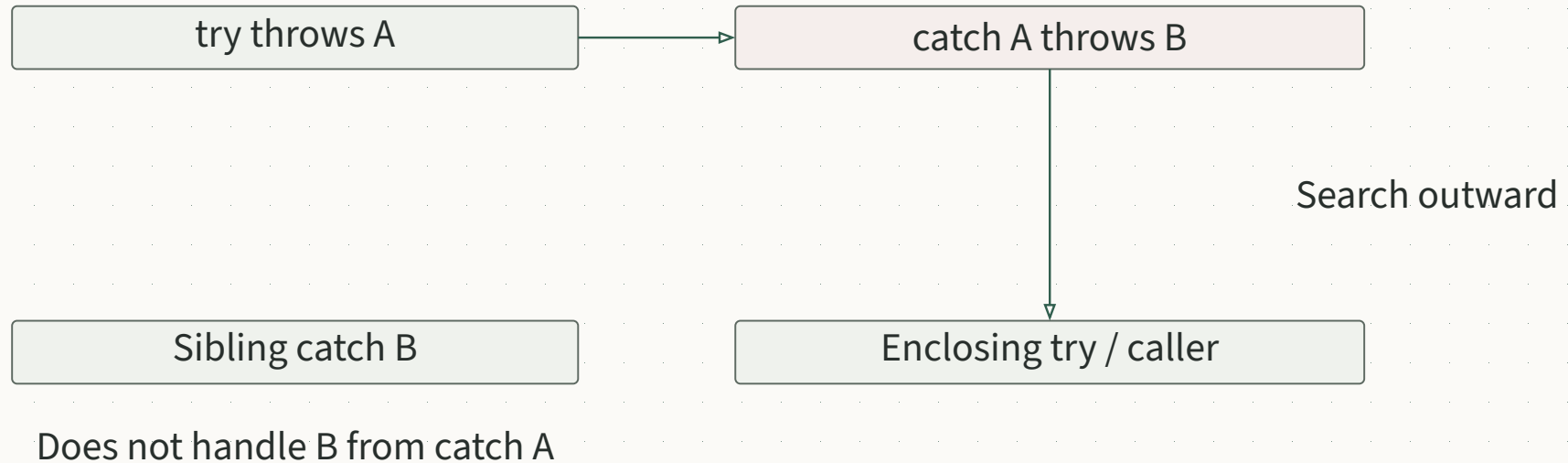
Is the new failure handled? Predict the output.

```
1  public static void main(String[] args) {  
2      int[] a = new int[4];  
3      try {  
4          a[4] = 0;  
5      } catch (ArrayIndexOutOfBoundsException e) {  
6          recursiveIncrease(7);  
7      } catch (StackOverflowError e) {  
8          System.out.println("Caught StackOverflowError");  
9      }  
10     System.out.println("Exiting main()");  
11 }
```

Java

Use recursiveIncrease from the previous example.

Sibling Catches Do Not Protect Each Other



Or put a nested try / catch around the failing code inside catch A

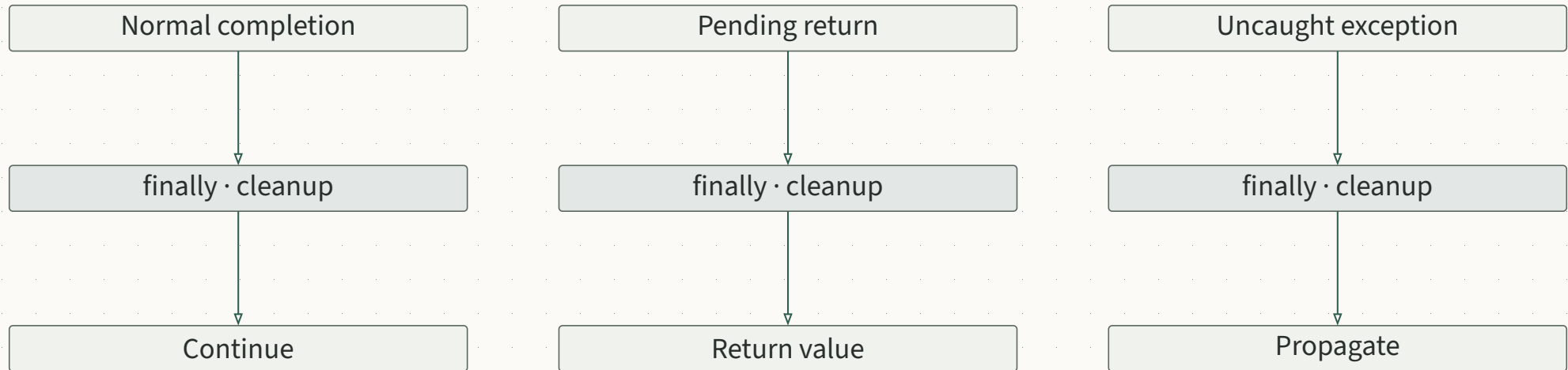
A Nested Handler

```
1  int[] a = new int[4];
2  try {
3      a[4] = 0;
4  } catch (ArrayIndexOutOfBoundsException e) {
5      try {
6          recursiveIncrease(7);
7      } catch (StackOverflowError inner) {
8          System.out.println("Caught inner StackOverflowError");
9      }
10 }
11 System.out.println("Continuing after the handler");
```

Java

In real code, fix the recursion rather than relying on this recovery.

Finally: Cleanup Before Leaving



Assumes finally completes normally · a return / throw there replaces the pending exit

Finally runs when control leaves try / catch, including on return or throw. It is not guaranteed if the JVM terminates or execution never leaves the block.

Write Cleanup Last

```
1 try {  
2     // Use a resource.  
3 } catch (SomeException e) {  
4     // Handle a specific failure.  
5 } finally {  
6     // Cleanup, e.g. close an opened resource.  
7 }
```

Java

SomeException is a placeholder for the relevant type. For closeable resources, prefer try-with-resources (next lecture).

Finally During Propagation

What is printed before the uncaught error? Use recursiveIncrease as before.

```
1  public static void main(String[] args) {  
2      int[] a = new int[4];  
3      try {  
4          a[4] = 0;  
5      } catch (ArrayIndexOutOfBoundsException e) {  
6          recursiveIncrease(7);  
7          System.out.println("Caught invalid index");  
8      } finally {  
9          System.out.println("Finally");  
10     }  
11     System.out.println("Exiting main()");  
12 }
```

Java

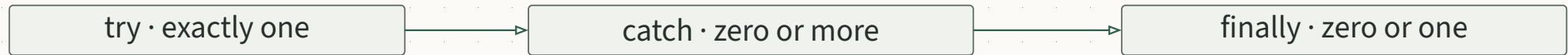
Finally Before a Return

Predict the output of `System.out.println("Ratio = " + getRatio(3, 0))`.

```
1  public static int getRatio(int a, int b) {  
2      int ratio = 0;  
3      try {  
4          ratio = a / b;  
5      } catch (ArithmeticException e) {  
6          System.out.println("Caught in getRatio()");  
7          return 0;  
8      } finally {  
9          System.out.println("Finally");  
10     }  
11     System.out.println("Exiting getRatio()");  
12     return ratio;  
13 }
```

Java

Valid Block Combinations



Ordinary try needs catch, finally, or both

Valid: try → catch · try → finally · try → catch → finally

Cleanup without a handler:

```
1 try {  
2     // May throw; no local recovery.  
3 } finally {  
4     // Cleanup before propagation.  
5 }
```

Java

5. Defining custom exceptions

Give a Domain Failure a Type

```
1 public static double squareRoot(double x) {  
2     return Math.sqrt(x);  
3 }
```

Java

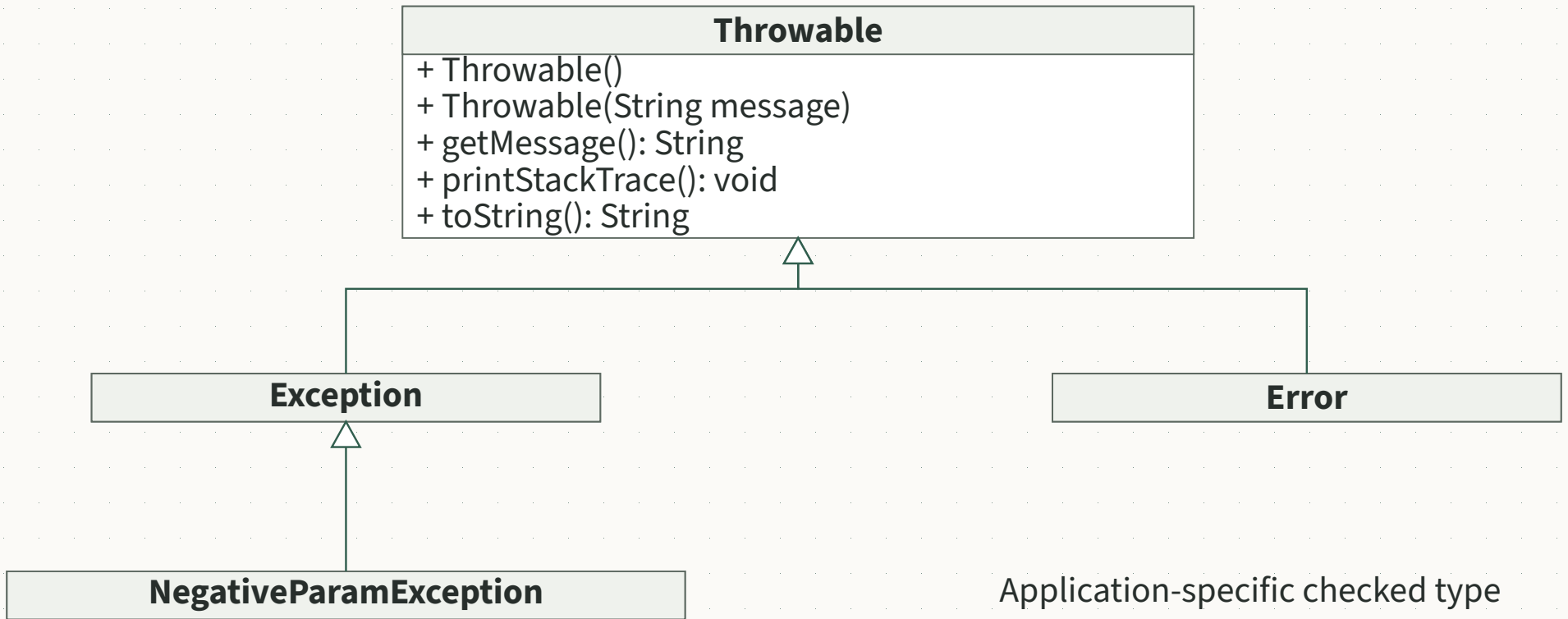
Math.sqrt returns NaN for negative input; this application should reject it.



Task 3

Define NegativeParamException and throw it when $x < 0$.

Extend an Existing Exception Class



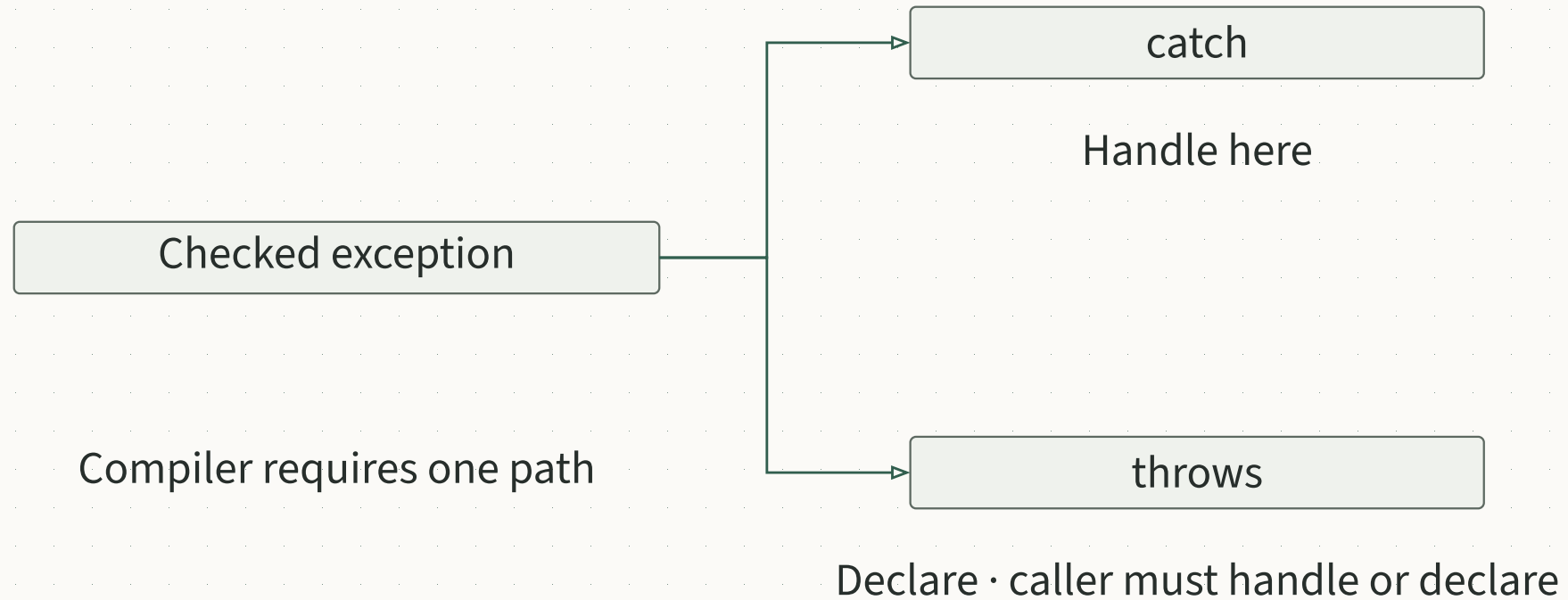
Why Does This Not Compile?

```
1  class NegativeParamException extends Exception {}
2
3  public class OwnException {
4      public static void main(String[] args) {
5          System.out.println(squareRoot(25.0));
6      }
7      public static double squareRoot(double x) {
8          if (x < 0.0) {
9              throw new NegativeParamException();
10         }
11         return Math.sqrt(x);
12     }
13 }
```

Java

The compiler reports an unhandled checked exception, even with positive input in main.

Checked Exceptions: Catch or Declare



Declare with throws

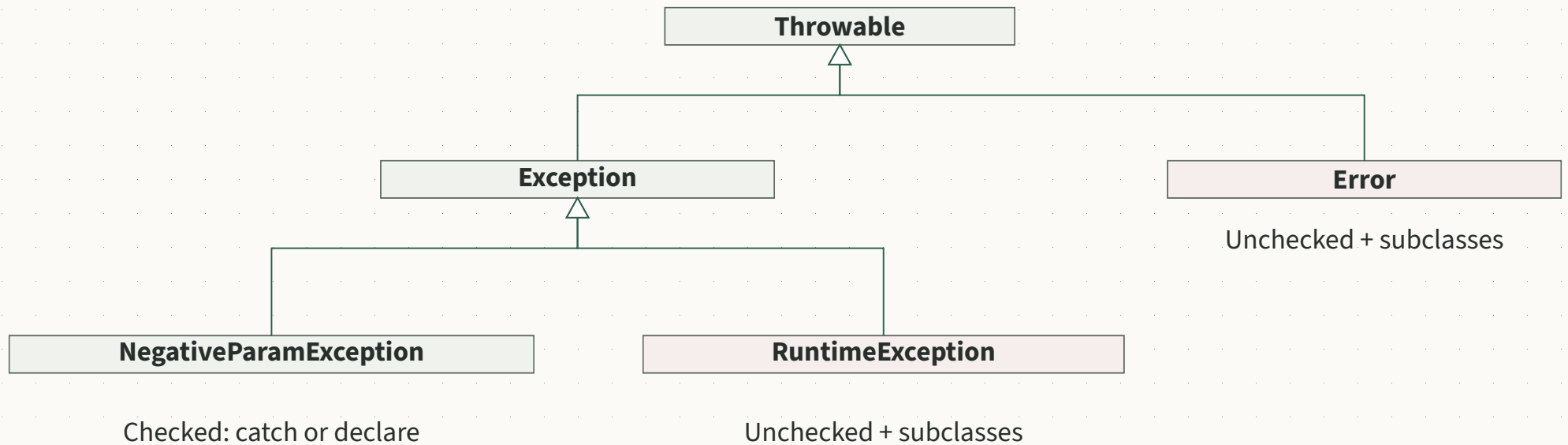
Use the same NegativeParamException class extending Exception.

```
1  public static void main(String[] args)
2      throws NegativeParamException {
3      System.out.println(squareRoot(25.0));
4  }
5  public static double squareRoot(double x)
6      throws NegativeParamException {
7      if (x < 0.0) {
8          throw new NegativeParamException();
9      }
10     return Math.sqrt(x);
11 }
```

Java

throws declares a possible failure; throw actually raises it. Declaring it in main does not handle it.

Checked and Unchecked Types



All other Throwable types are checked · unchecked does not mean harmless

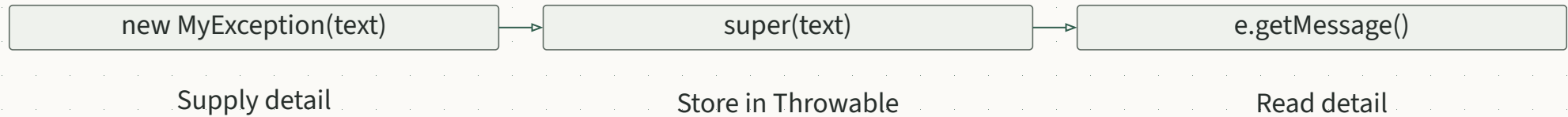
Alternative: An Unchecked Custom Exception

```
1  class NegativeParamException extends RuntimeException {}
2
3  public class OwnRuntimeException {
4      public static void main(String[] args) {
5          System.out.println(squareRoot(25.0));
6      }
7      public static double squareRoot(double x) {
8          if (x < 0.0) {
9              throw new NegativeParamException();
10         }
11         return Math.sqrt(x);
12     }
13 }
```

Java

No catch or throws declaration is required. A caller can still catch it.

Pass the Message to the Superclass



```
1 class MyException extends Exception {  
2     public MyException(String message) {  
3         super(message);  
4     }  
5 }
```

Java

Read the Custom Message

```
1 public static void main(String[] args) {  
2     try {  
3         throw new MyException("Invalid application state");  
4     } catch (MyException e) {  
5         System.out.println("Message: " + e.getMessage());  
6     }  
7 }
```

Java

Output: Message: Invalid application state

6. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0.
- Based on teaching material by Prof. Dr. Marc Hensel, supplied privately.
- Text, diagrams and layout adapted; the private source cannot be linked.